

Universidad Nacional Autónoma de México

Facultad de Ciencias

Ciencias de la Computación

Práctica 2: Expansión del analizador léxico

Compiladores

Semestre 2027-1

Equipo: DFGR

Integrantes: Flores Galeana Daniela - 321162342
Rivera Machuca Gabriel Eduardo - 321057608

Profesora: L. en C.C. Yessica Janeth Pablo Martínez

Ayudante: Pedro Yamil Salazar González

Fecha de entrega: 12 de septiembre de 2026

Índice

1. Objetivo

El objetivo principal de esta segunda práctica es expandir la infraestructura base del analizador léxico (lexer) del lenguaje MiniC construida en la entrega anterior. En esta fase, la meta es incorporar el reconocimiento de nuevos elementos léxicos: identificadores, palabras reservadas, literales booleanos y operadores compuestos (de dos caracteres). Adicionalmente, se implementó la capacidad de ignorar correctamente los comentarios y se corrigió el manejo de memoria del buffer léxico basándonos en la retroalimentación de la Práctica 1.

2. Descripción de la solución

2.1. Organización de módulos

Se mantuvo la estructura del proyecto separada en tres partes principales (`src/main.c`, `src/lexer/lexer.c` y `src/lexer/token.c`). Esta modularidad nos permitió agregar los nuevos tipos de tokens en `token.h` y encapsular la nueva lógica de clasificación dentro de `lexer.c` sin afectar la entrada y salida de datos del programa principal.

2.2. Representación de nuevos tokens

Se expandió el `enum TokenType` para incluir las nuevas categorías léxicas de MiniC:

- **Identificadores:** IDENTIFIER
- **Palabras reservadas:** INT, BOOL, IF, ELSE, WHILE, PRINT
- **Literales booleanos:** TRUE, FALSE
- **Operadores compuestos:** EQUAL (==), NOT_EQUAL (!=), LESS_EQUAL (<=), GREATER_EQUAL (>=), AND (&&), OR (||)

2.3. Manejo dinámico de lexemas (Corrección de Retroalimentación)

Para corregir la observación del ayudante en la Práctica 1, se eliminó por completo el uso de arreglos estáticos (como `char lexeme[100]`) al extraer números enteros e identificadores. Ahora se implementó una estrategia estrictamente dinámica:

```
1 size_t cap = 16;
2 size_t len = 0;
3 char *lexeme = malloc(cap);
4 // ... durante la lectura de caracteres:
5 if (len + 1 >= cap) {
6     cap *= 2;
7     char *temp = realloc(lexeme, cap);
8     // manejo de error si temp es NULL
9     lexeme = temp;
```

Esto garantiza que lexemas inusualmente largos no provoquen desbordamientos de buffer (buffer overflows) y optimiza el uso de la memoria.

2.4. Identificadores y palabras reservadas

Al detectar una letra o un guion bajo (`_`), el lexer extrae todos los caracteres alfanuméricos subsecuentes utilizando el buffer dinámico. Al finalizar la extracción del lexema, la cadena resultante se pasa por una función auxiliar `check_keyword` que compara el texto contra las palabras reservadas y booleanos utilizando `strcmp`. Si hay coincidencia, retorna el token correspondiente; en caso contrario, se clasifica como `IDENTIFIER`.

2.5. Operadores compuestos y Lookahead

Para reconocer operadores como `==` o `<=`, se implementó una lectura anticipada (*lookahead*). Cuando el lexer lee un `=` o un `<`, examina el siguiente carácter con `fgetc`. Si el carácter forma el operador compuesto, avanza la posición; de lo contrario, regresa el carácter no utilizado al flujo mediante `ungetc` y emite el símbolo simple.

2.6. Tratamiento de Comentarios

Se modificó la lógica al detectar el carácter de barra inclinada `/`:

- Si el siguiente carácter es `/`, se inicia un ciclo que consume todos los caracteres hasta encontrar un salto de línea o EOF.
- En caso contrario, regresa el carácter no utilizado al flujo mediante `ungetc` y emite un símbolo simple.

En caso de ser un comentario, el analizador no genera ningún token y simplemente continúa con la siguiente iteración del ciclo principal mediante un `continue`.

3. Resultados y pruebas

El programa fue validado utilizando el script de pruebas automatizado `run_public_tests.sh` proporcionado en la Práctica 2. Se pasó de forma exitosa los 13 casos públicos, incluyendo:

- `p01_identifiers.mc` y `p02_reserved_words.mc`: Verificación correcta de la tabla de palabras reservadas.
- `p05_compound_operators.mc`: Comprobación de que el *lookahead* funciona sin devorar caracteres erróneamente.
- `p07_comments.mc` y `p09_slash_and_comments.mc`: Ignoró correctamente los comentarios multilínea sin confundirlos con divisiones.

Adicionalmente, se ejecutó Valgrind para comprobar que el nuevo manejo dinámico de memoria (`malloc` / `realloc`) no generara fugas al liberar los tokens y los lexemas temporales.

4. Problemas encontrados

El principal reto de esta práctica fue sincronizar correctamente las variables de posición (`line` y `column`) al utilizar `ungetc`. Al leer un carácter extra para verificar un operador compuesto o un comentario que resultaba ser falso, retroceder en el archivo con `ungetc` requería cuidar de no duplicar incrementos de columna cuando ese carácter se volviera a leer en el siguiente ciclo. Además, la implementación inicial del `realloc` tenía un fallo donde perdíamos la referencia original si fallaba la asignación; esto se solventó usando un puntero temporal (`temp`) antes de reasignar.

5. Conclusiones

Esta práctica consolidó el funcionamiento del analizador léxico. Corregir el enfoque de memoria estática por uno completamente dinámico no solo solventó las críticas de la entrega pasada, sino que preparó nuestro compilador para casos de prueba más agresivos. La implementación de *lookahead* y el descarte de comentarios nos acercan significativamente al comportamiento de un compilador de producción, garantizando que el analizador sintáctico de las próximas fases reciba un flujo limpio y clasificado de tokens.

6. Uso de herramientas de Inteligencia Artificial

Para esta práctica se utilizó Gemini (Google) como herramienta de consulta y apoyo durante el desarrollo del trabajo.

- **Propósito:** Se utilizó principalmente para consultar dudas sobre algunos conceptos de C, manejo de memoria dinámica, `malloc`, `realloc`, `ungetc` y la lógica de *lookahead*.
- **Consultas realizadas:** Se hicieron preguntas relacionadas con el funcionamiento de algunas funciones y con posibles formas de implementar ciertas partes de la práctica. También se utilizó como apoyo para revisar algunos errores y entender mejor su causa.
- **Apoyo en el reporte:** La herramienta también se utilizó para revisar la ortografía y mejorar la redacción de algunas partes del reporte, así como para resolver dudas sobre la organización y formato en L^AT_EX.
- **Revisión manual:** Las sugerencias obtenidas fueron revisadas y adaptadas manualmente. La implementación final fue integrada con el código de la práctica y

se verificó su funcionamiento mediante la compilación y ejecución de las pruebas correspondientes.

7. Referencias

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2da ed.). Pearson.
- Fischer, C. N., Cytron, R. K., & LeBlanc, R. J. (2009). *Crafting a compiler*. Pearson.
- GeeksforGeeks. (s.f.). *Dynamic memory allocation in C using malloc(), calloc(), free() and realloc()*. <https://www.geeksforgeeks.org/dynamic-memory-allocation-in-c-using-malloc-calloc-free-and-realloc/>.
- GeeksforGeeks. (s.f.). *Introduction of lexical analysis*. <https://www.geeksforgeeks.org/compiler-design/introduction-of-lexical-analysis/>.
- International Organization for Standardization. (2018). *Information technology — Programming languages — C* (ISO/IEC 9899:2018).
- Kernighan, B. W., & Ritchie, D. M. (1988). *The C programming language* (2da ed.). Prentice Hall.
- Pablo Martínez, Y. J. (2026). *Compiladores_Practica2.pdf y GUIA_INTEGRACION_P2.md* [Material de clase]. Facultad de Ciencias, Universidad Nacional Autónoma de México.
- Valgrind Developers. (s.f.). *Valgrind user manual*. <https://valgrind.org/docs/manual/manual.html>